

รายงานผลการปฏิบัติงานระบบฐานข้อมูล

ผู้จัดทำ: นายปภักร ทองเจริญ

รหัสนักศึกษา: 6811011662008

โมดูลหลัก: U1 (Student Registration)

โครงการ: University Multi-Schema DB

1 ความรับผิดชอบและขอบเขตของระบบย่อย

5 Points

กระผมได้รับหน้าที่พัฒนาระบบ u1_reg (ทะเบียนนักศึกษา) ซึ่งเป็นแกนกลางสำคัญในการแปลงข้อมูลพื้นฐาน (Core Data) ให้กลายเป็นทรัพยากรที่พร้อมสำหรับการลงเรียน เช่น การจัดตั้งกลุ่มเรียน (Sections) การกำหนดวิชาที่ต้องผ่านก่อน และการจัดการระบบทุนการศึกษา

◆ ภาระงานระดับโมดูล (Module Scope)

- ออกแบบ Schema 5 ตารางหลักเพื่อรองรับกระบวนการงานทะเบียน
- สร้าง Data Population (Mock Data) สำหรับการทดสอบ
- วิเคราะห์และผูก Foreign Key เพื่อเชื่อมข้อมูลจาก Schema core

◆ การบูรณาการระดับกลุ่ม (Integration)

- พัฒนา Query สืบข้อมูลสถิติประยุกต์ข้ามระบบ (Cross-Schema)
- ออกแบบมาตรการป้องกันข้อมูลสูญหายด้วยกฎ CASCADE / SET NULL
- จำกัดสิทธิ์การเข้าถึงข้อมูลด้วยระบบ Role-Based Security

2 โครงสร้างสถาปัตยกรรมฐานข้อมูล (Database Schema)

15 Points

โครงสร้างตารางถูกปรับแต่งตามหลักการ 3rd Normal Form (3NF) เพื่อขจัดความซ้ำซ้อนของข้อมูล และรับประกันความถูกต้องแม่นยำ (Data Integrity)

ตาราง (Table)	คีย์หลัก (Primary Key)	คีย์รองที่อ้างอิงข้ามระบบ (Foreign Keys)
sections	section_id	course_id, semester_id, instructor_id, room_id (ดึงจากระบบ Core ทั้งหมด)
prerequisites	course_id + prereq_id	โยงกลับไปหา core.courses ทั้ง 2 คอลัมน์
advisors	student_id	student_id (นักศึกษา), instructor_id (อาจารย์ผู้สอน)
scholarships	scholarship_id	(เป็น Master Table ของระบบนี้)

💡 การวิเคราะห์จุดคอขวดของระบบ (Schema Refactoring Insight)

โครงสร้างตาราง advisors ในปัจจุบันใช้ student_id เป็น PK เพียงตัวเดียว ซึ่งมีข้อจำกัดเชิง Business Logic คือ "ไม่อนุญาตให้นักศึกษาเปลี่ยนที่ปรึกษาเลย" หากระบบมหาวิทยาลัยต้องยืดหยุ่นขึ้น ผมเสนอให้รับเพิ่ม semester_id หรือ effective_date เข้าเป็น Composite PK ครับ

3 การสืบค้นและประมวลผลข้อมูล (SQL Execution)

15 Points

ตัวอย่างที่ 1: รายงานสถิติการรับทุนการศึกษา (JOIN + GROUP BY + Aggregate)

การดึงข้อมูลจากตารางหลักของนักศึกษา ผสมผสานกับประวัติการรับทุน เพื่อสรุปยอดเงินและรายชื่อทุนที่ได้รับทั้งหมด

SQL QUERY: SCHOLARSHIP SUMMARY

```
SELECT
  s.student_id,
  CONCAT(s.first_name, ' ', s.last_name) AS student_name,
  COUNT(ss.scholarship_id) AS total_scholarships,
  SUM(sch.fund_amount) AS total_fund,
  GROUP_CONCAT(sch.scholarship_name SEPARATOR ', ') AS details
FROM core.students s
JOIN u1_reg.student_scholarships ss ON s.student_id = ss.student_id
JOIN u1_reg.scholarships sch ON ss.scholarship_id = sch.scholarship_id
GROUP BY s.student_id
ORDER BY total_fund DESC;
```

รหัสนักศึกษา	ชื่อ-สกุล	จำนวนทุน	ยอดเงินรวม (บาท)	รายละเอียดทุน
670001	ณัฐพงษ์ สุวรรณ	2	65,000.00	ทุนเรียนดี, ทุนยากจน
670002	วีรพล มั่นคง	1	50,000.00	ทุนเรียนดี
670005	อาทิตย์ ศรีสุข	1	15,000.00	ทุนยากจน

ตัวอย่างที่ 2: รายงานตารางสอนและกลุ่มเรียน (Multi-Table JOIN)

SQL QUERY: CLASS SCHEDULES

```
SELECT sec.section_id, c.course_name,  
       CONCAT(i.first_name, ' ', i.last_name) AS instructor,  
       r.room_name, sec.schedule_time, sec.max_seats  
FROM u1_reg.sections sec  
JOIN core.courses c ON sec.course_id = c.course_id  
JOIN core.instructors i ON sec.instructor_id = i.instructor_id  
LEFT JOIN core.rooms r ON sec.room_id = r.room_id;
```

4 นโยบายความปลอดภัยฐานข้อมูล (Database Security)

10 Points

เนื่องจากฐานข้อมูลนี้เป็นแบบ Multi-Schema การจำกัดสิทธิ์จึงใช้แนวทาง Role-Based Access Control ให้เจ้าหน้าที่ฝ่ายทะเบียนเข้าถึงได้เฉพาะตารางที่ตนรับผิดชอบ

DCL QUERY: GRANT & REVOKE

```
CREATE USER 'registrar_officer'@'%' IDENTIFIED BY 'SecureReg2026!';  
-- ให้สิทธิ์เต็มๆเฉพาะ Schema ทะเบียน  
GRANT SELECT, INSERT, UPDATE, DELETE ON u1_reg.* TO 'registrar_officer'@'%';  
-- ให้สิทธิ์อ่านอย่างเดียวในตารางพื้นฐานที่ต้องใช้อ้างอิง  
GRANT SELECT ON core.students TO 'registrar_officer'@'%';  
GRANT SELECT ON core.courses TO 'registrar_officer'@'%';
```

🚫 การทดสอบความปลอดภัย:

หาก registrar_officer พยายามไปรันคำสั่ง UPDATE u3_grade.student_scores SET score = 100; เพื่อแก้ไขเกรด ระบบจะบล็อกการทำงานทันทีและแสดงข้อความ **ERROR 1142: UPDATE command denied** ซึ่งพิสูจน์ได้ว่าระบบแบ่งแยกความรับผิดชอบสำเร็จ

5 การแก้ปัญหาเชิงเทคนิค (Troubleshooting)

5 Points

ปัญหาที่พบ (Issue)

วิธีการแก้ไข (Resolution)

ข้อมูลห้องเรียนหายพาเอาตาราง
สอนลบไปด้วย

(Cascade Delete Error)

เดิมที่กำหนด FK ของ room_id เป็น ON DELETE CASCADE เมื่อมีการลบห้องเรียนทิ้ง
ระบบเลยลบประวัติ Section ของเทอมก่อนๆ ทั้งหมด วิธีแก้คือเปลี่ยนเป็น ON
DELETE SET NULL ครับ เพื่อคงประวัติคลาสเรียนไว้แม้ห้องจะถูกลบ

ปัญหา Collation Mismatch ข้าม
ระบบ

เวลา JOIN ข้อมูลข้ามตาราง มักติด Error เรื่อง charset เนื่องจากบางตารางสร้างเป็น
utf8mb4_unicode_ci แต่อีกตารางเป็น utf8mb4_general_ci ต้องแก้โดยการ
เดิม COLLATE กำกับในประโยค JOIN

6 การเตรียมการนำเสนอ (Demonstration Guide)

10 Points

📢 บทนำเสนอ (Pitch Script):

"เรียนอาจารย์ประจำวิชาครับ ผมปลั่งกรดูแลโมดูล U1 ซึ่งเปรียบเสมือน 'ฝ่ายจัดเตรียม' ของมหาวิทยาลัยครับ ผมเป็นคน
เอาวิชา อาจารย์ และห้องเรียนมาแพ็คเป็น 'กลุ่มเรียน' เพื่อรอให้ระบบ U2 เอาไปให้เด็กลงทะเบียน การออกแบบของผม
เน้นความรัดกุมสูง เช่น ถ้าห้องเรียนโดนลบ ประวัติการสอนต้องไม่หาย และเจ้าหน้าที่ทะเบียนต้องไม่มีสิทธิ์แอบไปแก้
เกรดในระบบ U3 ครับ"

Q: ทำไมตาราง student_scholarships ต้องสร้าง Composite Primary Key ถึง 3 คอลัมน์?

เพื่อรองรับเหตุการณ์ที่ "นักศึกษาคนเดิม" สามารถรับ "ทุนการศึกษาประเภทเดิม" แต่เป็น "คนละภาคการศึกษา" ครับ หาก
เราใช้ PK แคร่ห้ส่นักศึกษากับรหัสทุน ระบบจะฟ้อง Duplicate Key Error ทันทีเวลาที่เด็กได้ทุนต่อเนื่องในเทอมหน้า การเอา
เทอมเข้ามาเป็น PK จึงแก้ปัญหานี้ได้ครับ

Q: ถ้าอยากเช็ควิชาเรียนก่อน (Prerequisite) ข้อนกันหลายชั้น โครงสร้างนี้รองรับไหม?

รองรับครับ เพราะตาราง prerequisites อ้างอิงกลับไป courses ทั้งสองฝั่ง ทำให้สามารถทำ Self-Join แบบ
Recursive เพื่อเช็ควิชาที่ต้องเรียนก่อนได้ลึกกี่ชั้นก็ได้ครับ